

ICU Chat Agent for Mortality Prediction and Medication Optimization

PHANIKISHORE CUNCHALA, University of Texas at Austin, USA

This project is an attempt to develop an intelligent chatbot system for intensive care settings that predicts patient mortality and survival while recommending medications to mitigate risk. The system is developed using pre-trained machine learning models on the MIMIC-III dataset and is designed to support clinical management of patients with hematological conditions. It provides decision support through a structured, clinician-oriented conversational interface, facilitating informed and efficient healthcare delivery. The proposed framework is generalizable and can be extended to broader clinical applications and disease domains

ACM Reference Format:

Phanikishore Cunchala. 2026. ICU Chat Agent for Mortality Prediction and Medication Optimization . 1, 1 (April 2026), 5 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

In ICU, there is an urgent need for a decision-based system that can make predictions about a patient's mortality and survival based on the features and suggest medications that can reduce the patient's mortality risk. These 2 functions will greatly be able to assist the healthcare professionals to better manage the available resources while providing quality health care to the patients. The recent advances in LLM provide a unique opportunity to develop an easy-to-use ML based chatbot system to address this need. This paper is an attempt to develop a chatbot that leverages ML based models that are pre-trained on publicly available MIMIC III Dataset. The system is for a very specific use-case. It tries to predict mortality rate and survivability as well as offer medications list that could reduce the mortality risk for female patients admitted to ICU that have been diagnosed for various blood diseases. The blood diseases are the diseases that are tagged with ICD-9 codes of 280 – 289. The chatbot aids and provides suggestions as a doctor persona in a simple question and answer format. Although a specific example has been demonstrated, the tool can be easily extended to other diseases and leverage other pretrained ML models. All the models have been developed ground up. The mortality and survivability model leverages Random Survival Forest with Hyper tuning while the medication prediction model leverages Random Forest Classifier. Both models are pre- trained on curated MIMIC III Datasets.

2 Related Work

The origins of Decision Trees trace back to decision tree methods such as **CART (Classification and Regression Trees)** developed

by Leo Breiman, Jerome Friedman, Richard Olshen, and Charles Stone (1984). These models introduced recursive partitioning for classification and regression tasks, forming the conceptual basis for ensemble methods.

Random Forest was formally introduced by Leo Breiman in 2001. It built upon earlier ideas such as **bagging (bootstrap aggregation)** and random feature selection to improve predictive accuracy and reduce overfitting. Some of the key innovations were

- Ensemble of multiple decision trees
- Random sampling of data (bootstrapping)
- Random subset of features at each split
- Aggregation via voting (classification) or averaging (regression)

Random Survival Forest (RSF) was introduced by Hemant Ishwaran and colleagues in 2008 as an extension of Random Forest for time-to-event (survival) data. Some key advancements:

- Handles **censored data** (patients lost to follow-up or not experiencing event)
- Uses survival-specific splitting rules (e.g., log-rank test)
- Outputs **survival curves** instead of single predictions
- Non-parametric alternative to classical models like Cox Proportional Hazards Model

The evolution of these machine learning methods provides the required foundation for the ICU Chatbot. For medication prediction components this ensemble approach is suitable because it captures complex relationships between patient characteristics, medications, and observed clinical outcomes. The survival prediction component extends this idea to time-to-event modeling, enabling the system to account for both mortality occurrence and survival duration after ICU admission. Thus, the proposed chatbot applies the progression of tree-based models to a clinically relevant ICU use case, supporting mortality estimation, survival analysis, and treatment recommendation for patients with blood related conditions.

3 Methodology

I followed the approach of first identifying the data from tables of MIMIC III dataset. This was then followed by running extraction queries to get the data for various cohorts. This was then followed by Data Cleaning and Feature Engineering to make the data model ready. I chose the Random Survival Forest ML model with hyper tuning to train the ML model to predict survival curves and risk estimates. For the Medication prediction model, I used the Random Forest Classifier to get a binary classification of mortality risk with the predicted list of medications that could lower the mortality risk. I implemented custom functions to make use of the pre-trained Mortality and medications prediction models. I also provided a doctor persona-based chat interface that ICU personnel can use to ask questions and obtain mortality risk, survival probability and medications list to review for a particular patient that aids in better decision making.

Author's Contact Information: Phanikishore Cunchala, pc32459@my.utexas.edu, University of Texas at Austin, Austin, Texas, USA.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM XXXX-XXXX/2026/4-ART

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

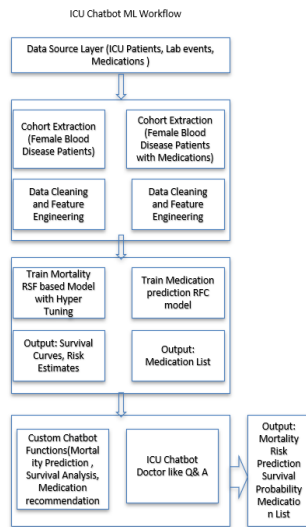


Fig. 1. Workflow

4 Data Generation

4.1 Extraction of Female Patients Cohort with Blood diseases

I identified the tables diagnoses_icd, icustays, patients, admissions, chartevents and labevents from MIMIC III Dataset to run a BigQuery SQL query to create a model ready CSV file that included ICU stay information, binary mortality label, first-24-hour heart rate features, and first-24-hour laboratory features like hemoglobin, hematocrit, and glucose. The Blood disease patients were identified as those who were diagnosed with ICD -9 codes 280 - 289.

4.2 Extraction of Female Patients Cohort with Blood diseases with medications list

I identified the tables icustays, patients, admissions, prescriptions from MIMIC III Dataset to run a BigQuery SQL query to create a model ready CSV file that included female ICU Patients with diagnoses code of ICD-9 (280 - 289) who were prescribed medications.

5 Mortality, Survival ML model and Feature Engineering

Mortality rate is a *single summary measure*: that tells the proportion of patients who die within a defined period (e.g., ICU mortality rate = deaths / total patients).

Survival analysis, on the other hand, is *time-aware*: and models **when** events (like death) occur, accounts for censoring, and produces survival probabilities over time (e.g., survival curves)

Mortality rate and survival analysis are complementary measures; while mortality rate summarizes overall outcome frequency, survival analysis captures the temporal dynamics of patient risk, enabling more informative and clinically relevant predictions.

After loading the female patients blood diseases cohort data, I calculated the mortality rate to understand the dataset distribution. Mortality rate was calculated as the difference between discharge

Model	AUC	Accuracy
Logistic	0.698	0.880
Random Forest	0.713	0.872
Gradient Boosting	0.739	0.880
XGBoost	0.704	0.875

Model	AUC	Accuracy
Logistic	0.698	0.880
Random Forest	0.713	0.872
Gradient Boosting	0.739	0.880
XGBoost	0.704	0.875

Fig. 2. Cox Model Output

time and admission time; Mean mortality rate was 13.2 % with mortality increasing with longer ICU stays.

My next step was to establish a baseline before moving to survival analysis modeling. I cleaned the data to handle missing values and drop non-numeric values. The dataset was an 80:20 split between training and testing. I used the following 4 models to get a binary outcome classification

- Logistic Regression
- Random Forest Classifier
- Gradient Boosting Classifier
- XGB Classifier.

The output was as follows:

Gradient Boosting was found to be the best in terms of AUC prediction and Accuracy.

5.1 Survival Analysis using Cox Proportional Hazards Model

The Cox Proportional Hazards Model is a semi-parametric method used in survival analysis to model the relationship between patient features and the risk of an event (e.g., death) over time, without assuming a specific baseline hazard function. It expresses the hazard at time t as a product of a baseline hazard and an exponential function of covariates, where each feature has a multiplicative effect on risk that remains constant over time (the assumption of proportional hazards).

$$h(t | X) = h_0(t) \exp(\beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p) \quad (1)$$

In this formulation, $h_0(t)$ is the baseline hazard, (X_i) are patient features, and (β_i) are coefficients estimated from the data; Exponentiation of the coefficients gives hazard ratios, quantifying how each variable increases or decreases the instantaneous risk of the event.

For preparation of survival dataset, I calculated 2 indicators. Event indicator (death = 1) and duration until event (death or Discharge) and added them.

The Cox proportional hazards model demonstrated strong predictive performance (C-index = 0.86) in estimating mortality risk among ICU patients. Heart rate metrics and hematocrit were identified as significant predictors, with increased mean heart rate and hematocrit associated with elevated mortality risk. Conversely, higher minimum heart rate was associated with reduced risk. Several variables, including hemoglobin and glucose, were not statistically significant, suggesting limited contribution to hazard estimation. The

model was globally significant based on the log-likelihood ratio test ($p < 0.001$), indicating that the selected covariates collectively improve survival prediction.

5.2 Survival Analysis using Random Survival Forest Model

Survival analysis using a Random Survival Forest (RSF) models time-to-event outcomes by building an ensemble of survival trees on bootstrapped samples, where each tree partitions patients using survival-based splits (e.g., log-rank test) and estimates a cumulative hazard function (CHF) over time. The final prediction aggregates these tree-level estimates to obtain a robust, time-dependent risk measure:

$$\hat{H}(t | X) = \frac{1}{B} \sum_{b=1}^B \hat{H}_b(t | X)$$

$$\hat{S}(t | X) = \exp(-\hat{H}(t | X))$$

where B is the number of trees and X represents patient features. This approach effectively handles censored data and captures complex nonlinear relationships, making it well-suited for clinical survival prediction tasks such as ICU mortality analysis.

I implemented a complete preprocessing and modeling pipeline for survival analysis using a Random Survival Forest model. I first removed invalid observations (e.g., non-positive durations) and ensured the event indicator was correctly formatted to improve data quality before modeling. I then split the dataset into training, validation, and test sets using stratification on the event variable to preserve the mortality distribution across splits. I constructed feature matrices by excluding identifiers and non-predictive variables, retaining only numeric features, and applied median imputation to handle missing values. To ensure consistency, I aligned feature columns across datasets and removed constant features to reduce redundancy. I created survival targets using structured time-to-event representations suitable for survival models. I performed hyperparameter tuning using grid search and selected the best model based on validation C-index, after which I retrained it on the combined training and validation data and evaluated its performance on the test set.

I split the dataset into training, validation, and test sets with balanced target distributions to ensure reliable model evaluation. During hyperparameter tuning, I evaluated multiple Random Survival Forest configurations using the validation C-index as the primary selection metric. I observed that increasing the number of trees and using larger leaf sizes improved generalization, as reflected in higher validation C-index values and stable out-of-bag (OOB) scores. The best-performing model achieved a validation C-index of 0.8239, indicating strong ability to rank patient risk over time. I then retrained the selected model on the combined training and validation data and evaluated it on the test set, achieving a C-index of 0.8634. The relatively small gap between training and test performance suggests that the model generalizes well with minimal overfitting.

As a last step, I saved the trained model to use in the chatbot app

6 Medication Prediction ML Model and Feature Engineering

For medication prediction model, I prepared dataset of female patients suffering from Blood diseases who were admitted to ICU and were administered various medications. The first step was to

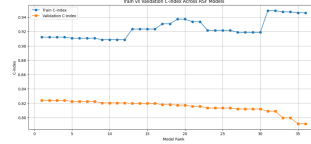


Fig. 3. Training Vs Validation C-Index across various models

clean the dataset by removing duplicates and normalizing medicine names. Used feature Engineering to convert medication list to binary columns. I used Random Forest Classifier as prediction model to come up with recommendation of medications that lowers predicted risk.

A **Random Forest Classifier** is an ensemble learning method that builds multiple decision trees on bootstrapped subsets of the data and combines their predictions to improve accuracy and reduce overfitting. Each tree makes an independent prediction, and the final class label is determined by majority voting across all trees, making the model robust and capable of capturing complex nonlinear patterns.

$$\hat{y} = \text{mode}\{h_1(X), h_2(X), \dots, h_B(X)\} \quad (2)$$

where $\hat{y}$ is the final predicted class, $h_b(X)$ is the prediction of the b^{th} decision tree for input features X , and B is the total number of trees in the forest. The operator $\text{mode}(\cdot)$ returns the most frequently occurring class label among all tree predictions, which forms the final output of the model.

The model achieved an **AUROC of 0.913**, indicating strong ability to distinguish between positive and negative classes across thresholds. The **AUPRC of 0.628** suggests moderate performance in identifying the positive class, particularly under class imbalance. Overall, the model demonstrated good ranking capability, though there is room to improve precision-recall performance.

As a last step, I saved the model to use it in the Chatbot app

7 ICU Chat Agent

This is the section where I unified the various CSV files, models and provided user friendly chat agent interface to help healthcare professionals in ICU.

As a reminder, the primary objective is to demonstrate the implementation of an ICU clinical decision-support chatbot that integrates pre-trained machine learning models with datasets for mortality prediction, survival analysis, and medication recommendation. The architecture emphasizes a structured pipeline in which machine learning outputs are generated first and subsequently interpreted through a conversational interface.

I designed the system to load large datasets and pre-trained models from Google Drive, ensuring scalability and efficient handling of clinical data. This setup allows seamless integration of both mortality and medication datasets, along with their corresponding trained models, which are stored externally due to their size constraints.

I used the mortality prediction module to compute patient-specific survival probabilities over a defined time horizon. This module dynamically calculates duration variables when not explicitly provided and constructs a structured feature vector from clinical inputs such

as heart rate, hematological parameters, and glucose levels. The model outputs survival probability, mortality probability, and a corresponding risk stratification category, thereby enabling interpretable clinical insights.

In parallel, I developed a medication recommendation module that leverages the medication model and evaluates the impact of candidate medications on predicted mortality risk. This module constructs a binary feature representation of patient medications and simulates the addition of new medications to estimate their effect on risk reduction. The recommendations are ranked based on the magnitude of predicted risk reduction, ensuring that the output remains clinically interpretable and prioritized for review.

To unify these components, I designed an ICU chatbot interface that invokes both the mortality and medication models and integrates their outputs into a structured context. This context is then passed to a large language model, which generates clinically coherent explanations in a doctor-like conversational format. Importantly, the system enforces a strict separation between prediction and explanation, ensuring that the language model does not independently infer medical decisions but instead interprets validated model outputs.

I further validated the system using real patient data extracted from CSV files derived from the dataset, demonstrating its applicability in realistic clinical scenarios. Multiple test cases were implemented to evaluate the chatbot's ability to process patient features, retrieve medication histories, and generate meaningful responses. These evaluations confirm that the system can effectively combine structured data processing with conversational AI to support clinical reasoning.

Finally, I incorporated explicit clinical safety guidelines and disclaimers within the system to ensure responsible usage. The chatbot is designed strictly as a decision-support tool, and all outputs are framed as recommendations for clinician review rather than prescriptive actions. This design choice reinforces the importance of human oversight and aligns the system with ethical and practical considerations in healthcare AI deployment.

8 Results

8.1 Mortality Model Performance

The mortality and survival analysis model performed to expectation. The hyper parameter tuning presented many challenges. After a lot of experimentation, I used the grid search strategy to systematically evaluate different combinations of Random Survival Forest parameters. I defined a parameter grid over key hyperparameters such as the number of trees, minimum samples required for splitting and leaf nodes, and the number of features considered at each split. The code iterates through all possible combinations using ParameterGrid, trains a model for each configuration, and computes performance metrics on both training and validation datasets. The **concordance index (C-index)** is used as the primary evaluation metric, as it measures the model's ability to correctly rank survival times, while the out-of-bag (OOB) score provides an additional estimate of generalization. For each configuration, results including C-index, OOB score, and computation time were stored for comparison. The best

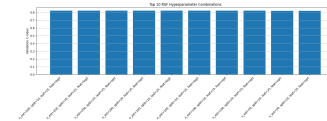


Fig. 4. Hyper param Tuning

Top tuning results:						
	n_estimators	min_samples_split	min_samples_leaf	max_features		
0	200	10	25	log2		
1	200	20	25	log2		
2	200	10	25	sqrt		
3	200	20	25	sqrt		
4	100	10	25	log2		
5	100	20	25	sqrt		
6	100	10	25	sqrt		
7	100	20	25	sqrt		
8	50	10	25	sqrt		
9	50	20	25	sqrt		

	train_cindex	val_cindex	oob_score	time_sec
0	0.912183	0.823925	0.838506	57.473828
1	0.912183	0.823925	0.838506	56.821122
2	0.912183	0.823925	0.838506	58.537086
3	0.912183	0.823925	0.838506	58.010806
4	0.910862	0.822448	0.834159	30.251143
5	0.910862	0.822448	0.834159	29.081028
6	0.910862	0.822448	0.834159	29.784779
7	0.910862	0.822448	0.834159	29.650819
8	0.908611	0.820303	0.829095	14.712693
9	0.908611	0.820303	0.829095	14.445779

Fig. 5. Tuning Results

model was selected based on the highest validation C-index, ensuring that the chosen hyperparameters generalize well to unseen data.

8.2 Limitation : Lack of Image Data in MIMIC III

As the project utilized MIMIC III Dataset, no image data was used. As a result the entire project is somewhat constrained as X-ray or CAT Scan were not considered while making the prediction.

8.3 Absence of User Interface

As the aim of the project was to demonstrate the development of a rudimentary ML based chat agent, no specific User Interface was implemented making this a strictly academic exploration only.

8.4 Tool Development Stage

This implementation is strictly in development / prototype stage only. Multiple enhancements would need to be added in terms of custom actions as well as more trained models.

9 Conclusion

When I started on this project, my main goal was to demonstrate that a useful chat agent that leverages ML models could be developed within a short time frame. The tool would try and provide evidence based answers to questions about medications that could reduce mortality risk as well address survivability risk questions.

The main highlight of this effort is the coming together of datasets, pre-trained ML models and chat style interface in a cohesive fashion thereby providing a template on which better tools can be built in the future.

10 References

- [1] Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and Regression Trees*. Wadsworth..
- [2] Breiman, L. (1996). *Bagging Predictors*. Machine Learning, 24(2), 123–140
- [3] Breiman, L. (2001).

Random Forests. Machine Learning, 45(1), 5–32.

[4] Ishwaran, H., Kogalur, U. B., Blackstone, E. H., & Lauer, M. S. (2008).

Random Survival Forests. Annals of Applied Statistics, 2(3), 841–860.

[5] scikit-learn Documentation – Random Forest

<https://scikit-learn.org>.

scikit-survivalDocumentation – Random Survival Forest

<https://scikit-survival.readthedocs.io>.

Leo Breiman (2001).

Random Forests.

Machine Learning, 45(1), 5–32.

The original paper introducing the Random Forest algorithm.

Leo Breiman(1996).

Bagging Predictors.

Machine Learning, 24(2), 123–140.

Introduces bootstrap aggregation (bagging), a key concept behind Random Forest.

Trevor Hastie, Robert Tibshirani, & Jerome Friedman (2009).

The Elements of Statistical Learning.

Chapter on ensemble methods and Random Forests.

Gareth James et al. (2013).

An Introduction to Statistical Learning.

Provides an accessible explanation of Random Forest and ensemble learning